JAVA 2

Module FX - seconde partie

Propriétés et architecture

Au programme

1. Les propriétés
2. Les bindings
3. MVC
4. Le FXML

1. Une valeur qui prévient



Réagir à un geste, ou à une valeur

Un gestionnaire réagit à un geste de l'utilisateur

Une valeur peut changer sans aucun geste : un collage, un calcul, le programme lui-même

Ce qu'on veut souvent savoir, c'est que la valeur a changé

JavaFX rend les valeurs observables : ce sont les propriétés

Une propriété est un objet

Un champ ordinaire a un accesseur et un mutateur

Une propriété JavaFX y ajoute un troisième accès, qui retourne l'objet lui-même

Il existe une classe par type : IntegerProperty, DoubleProperty, StringProperty, BooleanProperty

```
private final IntegerProperty intensite = new SimpleIntegerProperty(50);

public int getIntensite() { return intensite.get(); }

public void setIntensite(int valeur) { intensite.set(valeur); }

public IntegerProperty intensiteProperty() { return intensite; }
```

S'abonner à un changement

La méthode `addListener` abonne un `ChangeListener` à la propriété

Il reçoit trois choses : la propriété, l'ancienne valeur, la nouvelle

Il n'est appelé qu'aux changements, jamais à l'abonnement

```
intensite.addListener((propriete, ancienne, nouvelle) ->
        System.out.println(ancienne + " -> " + nouvelle));
```

```
intensite.set(80);   // affiche 50 -> 80
intensite.set(80);   // n'affiche rien : la valeur n'a pas change
```

Les contrôles en ont aussi

Presque tout ce qu'affiche un contrôle est une propriété

Une `CheckBox` a une `selectedProperty`, un `Label` une `textProperty`

On s'y abonne exactement comme à la sienne

```
CheckBox veilleuse = new CheckBox("Veilleuse");

veilleuse.selectedProperty().addListener(
        (propriete, avant, apres) -> passerEnVeilleuse(apres));
```

Exercices



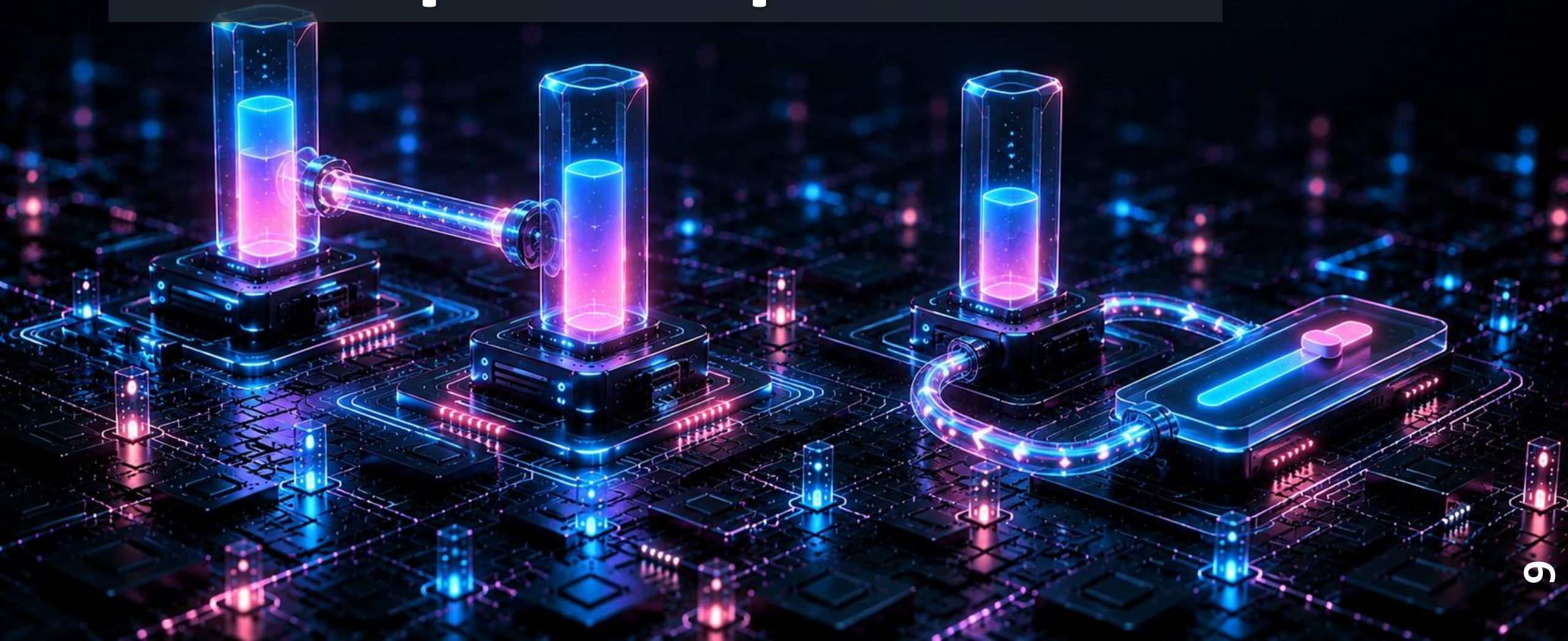
FX.6 Écouter une propriété

fascicule FX

FX.7 Un Slider et un Label

fascicule FX

2. Lier plutôt qu'écouter



Une dépendance déclarée

La méthode `bind` lie une propriété à une valeur observable

La propriété suit cette valeur, à chaque changement, pour toujours

Rien à afficher au départ : le lien s'applique dès qu'il est posé

Une propriété liée ne s'écrit plus que par son lien

```
Label affichage = new Label();

affichage.textProperty().bind(intensite.asString());

intensite.set(80);   // le libelle affiche 80, sans autre ligne
```

Des valeurs qui se calculent

Sur	La méthode	Produit
un nombre	`add`, `multiply`	un nombre
un nombre	`greaterThan`, `lessThan`	un booléen
un nombre	`asString`	un texte
un texte	`length`, `isEmpty`	un nombre, un booléen
un booléen	`not`, `and`, `or`	un booléen

Dans les deux sens

La méthode `bind` fait suivre une propriété, dans un seul sens

La méthode `bindBidirectional` garde deux propriétés égales, dans les deux sens

Chacune peut alors être modifiée, et l'autre suit

```
Slider curseur = new Slider(0, 100, 50);

curseur.valueProperty().bindBidirectional(reglage);

reglage.set(75);   // le curseur se place sur 75
// et si l'utilisateur deplace le curseur, reglage le suit
```

Exercices



FX.8 Lier plutôt qu'écouter

fascicule FX

3. Séparer l'écran du calcul



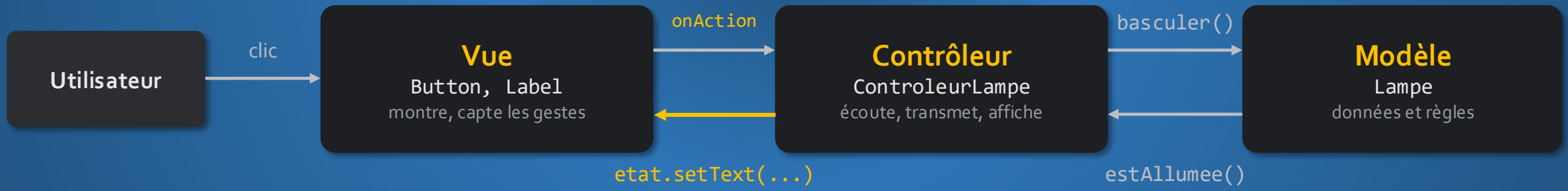
Trois responsabilités

Le modèle tient les données et les règles, et ignore tout de l'écran

La vue montre, et ne décide de rien

Le contrôleur écoute la vue, interroge le modèle, met la vue à jour

Le modèle se teste par `JUnit` ; une fenêtre ne se teste pas



Qui connaît qui

Le contrôleur connaît le modèle et la vue

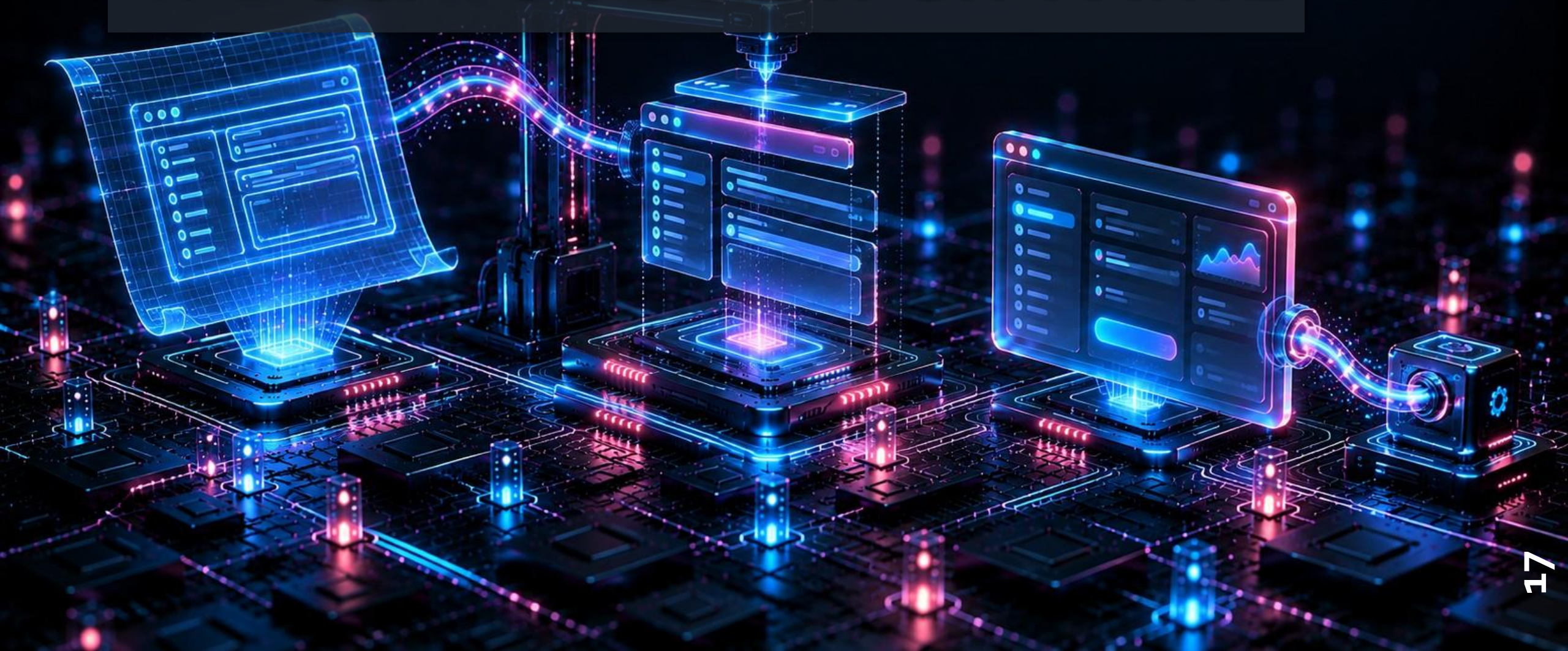
Le modèle ne connaît personne

La vue ne connaît pas le modèle

```java
public class Lampe {                    // le modele, sans JavaFX
    private boolean allumee;
    public void basculer() { allumee = !allumee; }
    public boolean estAllumee() { return allumee; }
}
```

```java
private void basculer() {               // dans le controleur
    lampe.basculer();
    etat.setText(lampe.estAllumee() ? "Allumee" : "Eteinte");
}
```

4. Décrire l'écran en FXML



Écrire l'écran, ou le décrire

Jusqu'ici, la vue était du code Java, dans `start`

Elle peut aussi se décrire dans un fichier `.fxml`, à part

Scene Builder dessine ce fichier à la souris

Le FXML n'est pas à l'examen

Un élément, un objet

Un élément crée un objet de la classe du même nom

Un attribut appelle le mutateur du même nom : `spacing` appelle `setSpacing`

Les éléments imbriqués deviennent les enfants

Les classes s'importent, comme en Java

```
<?import javafx.scene.control.Button?>
<?import javafx.scene.control.Label?>
<?import javafx.scene.layout.VBox?>

<VBox spacing="10" alignment="CENTER">
    <Label text="Eteinte"/>
    <Button text="Interrupteur"/>
</VBox>
```

Code FXML, une spécialisation de XML…

Charger une vue

Le fichier se range dans `src/main/resources`, dans le même package que la classe

Un `FXMLLoader` le lit et retourne la racine qu'il décrit

Le reste ne change pas : une scène, une fenêtre

```
@Override
public void start(Stage fenetre) throws IOException {
    FXMLLoader chargeur = new FXMLLoader(
            ApplicationLampe.class.getResource("lampe.fxml"));

    fenetre.setScene(new Scene(chargeur.load()));
    fenetre.show();
}
```

La vue et son contrôleur

L'attribut `fx:controller` nomme le contrôleur

Un `fx:id` remplit le champ `@FXML` de même nom

Un `onAction` appelle la méthode `@FXML` désignée

```
public class ControleurLampe {
    private final Lampe lampe = new Lampe();

    @FXML private Label etat;

    @FXML
    private void basculer(ActionEvent evenement) {
        lampe.basculer();
        etat.setText(lampe.estAllumee() ? "Allumee" : "Eteinte");
    }
}
```

Exercices



FX.9 Une calculatrice, en MVC

fascicule FX

Pour aller plus loin

Le démineur : un projet `JavaFX` complet, facultatif

Vous écrivez la grille et ses règles ; la fenêtre est fournie

Onze tests `JUnit` vérifient votre grille, sans ouvrir de fenêtre

Le fil rouge



G.34 Des boutons qui savent quand se taire

fil rouge GameHub

Relire la question de fin du deck FX1